MODULE 1 · UNIT 1.1 — FINE-TUNING FOR TASK-SPECIFIC OPTIMIZATION

Introduction to Fine-Tuning LLMs

From a General-Purpose LLM to a Task-Specific LLM



Lecture 2 · 75–90 minutes

IN THIS LECTURE

01 Why Fine-Tuning?
02 Pre-Training vs Fine-Tuning
03 Prompting vs Fine-Tuning
04 RAG vs Fine-Tuning
05 Fine-Tuning Dataset
06 Supervised Fine-Tuning
07 Full Fine-Tuning
08 Introduction to PEFT

SECTION 01 / 04

From Base Model to Specialized LLM

> A base model knows **language**. Your application needs **behavior**.

SECTION 01 · BASE MODELS

Where We Left Off — and What Is a Base Model?

QUICK REVISION FROM LECTURE 1

Raw Text → Tokenizer → Tokens → Embeddings → Transformer → Model Parameters → Next-Token Prediction → Generated Response

Massive General Dataset → Pre-Training → Base LLM

DEFINITION

A **base model** is an LLM that has learned general language patterns from a very large corpus.

IT MAY UNDERSTAND

- Language
- Grammar
- General facts
- Common concepts
- Code patterns
- Relationships between concepts

...but it is not necessarily optimized for your specific task.

THE BANK EXAMPLE

It can answer knowledge questions:

- What is a bank?
- What is a loan?
- What is a debit card?

CUSTOMER QUERY: My ATM card was stolen.

REQUIRED OUTPUT — STRUCTURED

- Intent: LOST_CARD
- Risk: HIGH
- Action: BLOCK_CARD
- Department: CARD_SERVICES

A base model might not consistently produce this exact structure — **that is where specialization becomes necessary.**

KNOWLEDGE

What information does the model know?

- What is a credit card?

BEHAVIOR

How should the model respond? Classify a complaint into:

- CARD_LOST
- CARD_BLOCKED
- FRAUD
- PAYMENT_FAILURE

Fine-tuning is especially useful to change or standardize behavior — always produce JSON, follow company terminology, classify messages, generate a specific style, follow specialized instructions.

RAG is often better for supplying external or frequently changing knowledge.

SECTION 01 · CORE DEFINITION

What Is Fine-Tuning?

DEFINITION

Fine-tuning is the process of taking an **already pre-trained model** and continuing its training using a **smaller, task-specific or domain-specific dataset**.

THE PIPELINE — FROM GENERAL TO SPECIALIZED

■ FINE-TUNING STAGES

STAGE 1 General Dataset → STAGE 2 Pre-Training → STAGE 3 Base LLM → STAGE 4 Task-Specific Dataset → STAGE 5 Fine-Tuning → STAGE 6 Specialized LLM

> *You already know English. Now learn how I want you to perform this particular task.*

— the intuition of fine-tuning in one sentence

HUMAN LEARNING — THE SAME IDEA

General Education → Broad Knowledge → Specialization

Doctor → Cardiology | Engineer → Cybersecurity | Lawyer → Corporate Law

A cardiologist does not relearn English, biology or human anatomy from zero — **fine-tuning follows the same conceptual idea.**

Fine-tuning modifies some or all trainable model parameters (weights & biases) | Task Data → Prediction → Loss → Backpropagation → Parameter Update

SECTION 01 · WHY FINE-TUNE

Why Fine-Tune? Task-Specific Optimization

DEFINITION | Task-specific optimization means **adapting an LLM so that it performs better on a clearly defined application task**.

WHAT FINE-TUNING DELIVERS

- Task specialization
- Domain-specific terminology
- Consistent output format
- Consistent response style
- Classification
- Information extraction
- Summarization style
- Instruction-following behavior
- Reduced dependence on long prompts

EXAMPLE — UNIVERSITY HELPDESK

The university receives queries such as:

- `> My examination form is not opening.`
- `> My fee payment failed.`
- `> How do I get my transcript?`
- `> I lost my ID card.`

DESIRED MODEL

`Query` → `LLM` → `Department / Category / Priority / Suggested Action`

WORKED EXAMPLE

"My semester fee was deducted from my bank account but the portal still shows unpaid."

↓

- Department: **Accounts**
- Category: **Payment Reconciliation**
- Priority: **High**
- Action: **Verify transaction reference**

A generic chatbot may answer conversationally — but the university requires a **structured workflow output**. A strong candidate for fine-tuning.

GOOD TASK — MEASURABLE

"Classify student complaints into 12 departments"

VAGUE OBJECTIVE

"Make my model better"

SECTION 01 · PLACING FINE-TUNING IN CONTEXT

Pre-Training vs Fine-Tuning vs Training From Scratch

	PRE-TRAINING	FINE-TUNING
STARTING POINT	Starts from random/initial model state	Starts from pre-trained model
DATA	Huge general dataset	Smaller specialized dataset
LEARNING FOCUS	Learns broad language patterns	Learns task-specific behavior
COST	Extremely expensive	Much less expensive
DURATION	Long training process	Shorter training process
RESULT	Creates foundation/base model	Creates specialized model

TRAIN FROM SCRATCH

`Random/Initial Parameters` → `Huge Dataset` → `Massive Compute` → `Model`

FINE-TUNING

`Existing Model` → `Small Specialized Dataset` → `Additional Training` → `Specialized Model`

You need an LLM for your college helpdesk — which is practical?

`A. Train a new billion-parameter model from scratch`

`B. Take an existing model and adapt it` → `ANSWER: B`

Analogy: **Pre-Training = bachelor's / general education · Fine-Tuning = specialization** (use for conceptual clarity only — not literal)

02

SECTION 02 / 04

Prompting, RAG & Fine-Tuning

Fine-tuning is not always the first solution — **match the lever to the problem.**

THE THREE LEVERS

Prompting | RAG | Fine-Tuning

SECTION 02 · ALTERNATIVE #1

Can We Avoid Fine-Tuning?

⚠ Fine-tuning is not always the first solution.

Before fine-tuning, ask: can the task be solved using a good prompt?

ALTERNATIVES: Prompt Engineering | RAG | Fine-Tuning | RAG + Fine-Tuning

QUICK EXAMPLE

Want `"Respond in exactly three bullet points."` ?

Do not train a model — simply instruct:

```
"Answer using exactly three concise bullet points."
```

DEFINITION — PROMPT ENGINEERING

Prompt engineering means **designing effective instructions and context** for an LLM **without modifying model parameters**.

<> WORKED PROMPT

```
"You are a university helpdesk assistant. Classify the following query into:
  1. Examination  2. Accounts  3. Library  4. Admission
Return only the department name.
Query: My fee payment is not reflected."
```

→ EXPECTED **Accounts**

WHAT CHANGES INSIDE THE MODEL?

Before Prompt → 🔒 Same Parameters

After Prompt → 🔒 Same Parameters

Nothing inside the model has changed; we only changed the input.

SECTION 02 · COMPARISON

Prompting vs Fine-Tuning

PROMPT ENGINEERING | Change Instructions → Same Model Parameters

FINE-TUNING | Training Examples → Update Model Parameters

ANALOGY

Prompting = giving a person detailed instructions *every time* they perform a task.

Fine-tuning = training the person so that they *already know the procedure*.

THE SAME TASK, TWO WAYS

PROMPT APPROACH — every request contains

```
"You are a banking assistant. Classify the query. Use only these categories… Return JSON… Follow these rules…"
```

FINE-TUNED APPROACH — simply send `"My card was stolen."` →

```
{"intent": "lost_card", "priority": "high"}
```

Fine-tuned models still use prompts — the desired behavior is more deeply learned. Do not say fine-tuning removes the need for prompts.

WHEN IS PROMPTING ENOUGH?

- Task is simple
- Existing model already performs well
- Requirements change frequently
- Few examples are available
- Rapid prototyping is required
- No training infrastructure is available

Golden rule: always establish a prompt-only baseline before spending resources on fine-tuning.

SECTION 02 · PROMPTING, RAG & FINE-TUNING

What Is RAG?

DEFINITION

Retrieval-Augmented Generation — RAG allows an LLM to use relevant external information retrieved at inference time.

User Question → Search / Retrieval → Relevant Documents → LLM → Answer

Example — frequently changing information

"What is the last date for university examination registration?" — this information may change every semester, and fine-tuning the model every time the date changes is inefficient.

BETTER

Question → Retrieve Current Examination Circular → Send Relevant Section to LLM → Answer

Update example

POLICY POLICY DOC: "The examination form submission deadline is 15 September."

USER "When is my exam form deadline?" → retrieve policy → LLM

ANSWER "Your examination form deadline is 15 September."

Tomorrow the university changes the deadline to 20 September — what should we update?

Only the document / database — we do not necessarily need to fine-tune the LLM again.

FINE-TUNING

Fine-Tuning changes HOW the model behaves

RAG

RAG changes WHAT information the model sees for the current request

RAG keeps the model unchanged and provides relevant external context.

SECTION 02 · PROMPTING, RAG & FINE-TUNING

Prompting vs RAG vs Fine-Tuning

7 REQUIREMENTS × 3 TECHNIQUES

Requirement	Prompting	RAG	Fine-Tuning
Change instructions	✓	—	—
Provide external knowledge	—	✓	—
Frequently changing facts	—	✓	—
Change response behavior	—	—	✓
Learn specialized format	Sometimes	—	✓
Quick prototype	✓	—	—
Consistent task-specific behavior	—	—	✓

✓ = the technique directly addresses this requirement

CASE A

"Write the answer formally."

USE **Prompting**

CASE B

"Answer using our latest 2026 university regulations."

USE **RAG**

CASE C

"Every complaint must be classified into our 15 internal categories."

CONSIDER **Fine-Tuning**

RAG + Fine-Tuning Together — and How to Decide

They are not competitors. They can be combined.

User Query → Retriever → Current Information → Fine-Tuned LLM → Domain-Specific Response

RAG = Current Knowledge

Fine-Tuning = Specialized Behavior

RAG provides

- Current fee circular
- Examination dates
- Latest rules
- Academic calendar

model stays unchanged

Fine-tuning provides

- Required response style
- Department classification
- Escalation behavior
- Structured output

model weights are updated

Decision tree

What is the problem?

Instruction problem	Knowledge problem	Behavior problem
Prompt	RAG	Fine-Tuning

Knowledge + Specialized Behavior
→ RAG + Fine-Tuning

"The company manual changes every month." → RAG

"Model must always output a fixed classification schema." → Fine-tuning

"Answer in Marathi." → Try prompt engineering first

SECTION 02 · DECISION GUIDE

When Should We Fine-Tune — and When Not?

Fine-tuning becomes attractive when

- Prompt-only performance is insufficient
- Consistent behavior is required
- Specialized output format is important
- Domain terminology is unusual
- Task repeats at high volume
- Training examples are available
- Smaller model needs stronger task performance
- Long repetitive prompts are undesirable

Avoid jumping directly to fine-tuning when

- Requirements are unclear
- Dataset quality is poor
- Only a few unreliable examples exist
- Information changes daily
- Prompting already works
- RAG solves the knowledge problem
- There is no evaluation benchmark

No Evaluation → No Meaningful Fine-Tuning

Because if we don't know the baseline performance, we cannot prove that fine-tuning improved anything.

03 / 04 SECTION

Fine-Tuning Data & Supervised Fine-Tuning

Garbage in, garbage out — the dataset decides the model.

DATA QUALITY · EXAMPLE FORMATS · SFT · SPLITS · LEAKAGE

SECTION 03 · FINE-TUNING DATA & SUPERVISED FINE-TUNING

Fine-Tuning Is a Data Problem

⚠ Garbage In → Garbage Out.

DATA QUALITY DECIDES THE OUTCOME

Poor Data → Poor Fine-Tuning → Poor Model

High-Quality Data → Better Fine-Tuning → Better Task Performance

Anatomy of a fine-tuning example

Instruction + Input + Expected Output

INSTRUCTION	INPUT	EXPECTED OUTPUT
"Classify the customer complaint."	"My debit card has been stolen."	LOST_CARD

Mislabel example

"My ATM card was stolen." incorrectly labeled `PAYMENT_FAILURE` → the model learns the wrong relationship.

A sophisticated model cannot compensate reliably for systematically bad labels.

CUSTOMER COMPLAINT	INTERNAL CATEGORY
My card was stolen	LOST_CARD
Transfer is still pending	TRANSFER_PENDING
ATM deducted money but gave no cash	ATM_CASH_FAILURE
I forgot my password	LOGIN_ISSUE
I want to close my account	ACCOUNT_CLOSURE

The model learns the organization's mapping `Natural Language → Internal Category` — that is task-specific optimization.

Fine-tuning is not limited to classification — Customer Response Generation

the model learns → tone · structure · response policy · wording patterns

INPUT "My parcel arrived damaged." → EXPECTED RESPONSE "We apologize for the inconvenience. Please upload photographs of the damaged item and packaging through the Returns section."

SECTION 03 · FINE-TUNING DATA & SUPERVISED FINE-TUNING

Instruction Fine-Tuning, Chat Format & SFT

Instruction fine-tuning

Trains a model using examples containing instructions and desired responses — instead of one single text-continuation pattern, we explicitly expose the model to **Instruction → Desired behavior**.

```
INSTRUCTION   "Summarize the following paragraph in one sentence."
INPUT         [Paragraph]
RESPONSE      [Correct Summary]
```

```
INSTRUCTION   "Extract the student's roll number."
INPUT         "My name is Rahul and my roll number is 128."
RESPONSE      128
```

Chat format

common for conversational / instruction models

SYSTEM	You are a university helpdesk assistant.
USER	My tuition payment is not reflected.
ASSISTANT	Department: Accounts · Priority: High · Action: Verify transaction.

System = defines high-level model behavior

User = provides user request

Assistant = provides expected response

Supervised Fine-Tuning (SFT)

SFT trains an existing LLM using input–target examples where the desired answer is known.

WHY "SUPERVISED"?

The model receives a target / correct response during training — e.g. Input "This movie is excellent." → **Label: Positive**

One training example

```
INPUT     "My hall ticket is not available."
EXPECTED  Department: Examination · Issue: Hall Ticket · Priority: High
```

1. Input ↓
2. Model ↓
3. Generated Prediction ↓
4. Compare with Expected Output ↓
5. Loss ↓
6. Backpropagation ↓
7. Parameter Update

Repeated over many examples → the model's behavior changes.

Dataset Quality: Consistency, Diversity, Splits & Leakage

Consistency matters

inconsistent labels confuse the model

"My card is stolen."	LOST_CARD
"My debit card has been stolen."	FRAUD
"I lost my ATM card."	CARD_MISSING

Better: all three → LOST_CARD

Fine-tuning does not magically resolve poorly defined classes — define annotation guidelines before training.

Diversity matters

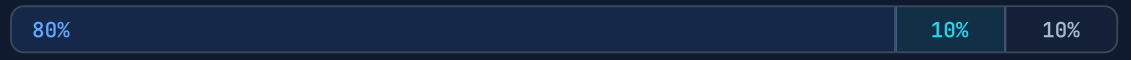
same intention, many phrasings

"My payment failed." "Payment isn't going through." "Why can't I pay?"
"The transaction keeps failing." "Unable to complete payment."

→ all → PAYMENT_FAILURE

Real users will not use exactly the same words as the training data — learn the concept, not memorize a sentence.

Train / Validation / Test



Training (Learn) = learn parameters

Validation (Tune) = compare configurations, select checkpoints, choose hyperparameters

Test (Final check) = only for final unbiased evaluation

Example 80% / 10% / 10% — the percentages are not universal rules.

Unseen data & leakage

A student memorizes 100 practice questions — the exam contains exactly those same 100 → 100% proves nothing.

Training Data Performance ≠ Generalization Performance — a model may simply overfit.

Data leakage — information from validation/test data improperly influences training. Bad: "My card has been stolen." in BOTH training and test.

- ✓ remove duplicates before splitting
- ✓ check near-duplicates
- ✓ avoid using test data for hyperparameter selection — particularly important in research papers

SECTION 04 / 04 — MODULE 1 · UNIT 1.1 · LECTURE 2

Workflow, Full Fine-Tuning & the Road to PEFT

From a 13-step experimental pipeline to one expensive question.

- The 13-Step Workflow
- Full Fine-Tuning & Its Cost
- The Road to PEFT

01 — 02 — 03 — 04

SECTION 04 · FINE-TUNING IN PRACTICE

The Fine-Tuning Workflow

13 steps · one experimental pipeline

1 Define Task → 2 Build Baseline → 3 Collect Dataset → 4 Clean Dataset → 5 Split Dataset → 6 Format Examples → 7 Tokenize
8 Load Base Model → 9 Configure Training → 10 Fine-Tune → 11 Validate → 12 Test → 13 Deploy

> *Fine-tuning is not simply calling `train()`. It is an experimental pipeline.*

1 Define the Task

BAD OBJECTIVE

"Create an intelligent university chatbot."

BETTER

"Classify student helpdesk queries into 10 predefined departments."

EVEN BETTER

"Achieve high macro-F1 on 10 student-query categories while returning responses within an acceptable latency."

A production task needs measurable success criteria.

2 Establish a Baseline

Base Model + Prompt → Evaluate → Fine-Tuned Model → Evaluate Again

Record: `Accuracy` / `F1` · response quality · format correctness · latency · memory · cost

base 85% → tuned 86% Was fine-tuning worth substantial engineering cost? Maybe not.

base 65% → tuned 91% Now the improvement is significant.

Evaluate the same way, before and after.

3 Select a Base Model

model capability · parameter count · context length · license · hardware requirements · supported language · domain performance · fine-tuning support · deployment target

3B → 94%	task accuracy
70B → 95%	costs dramatically more to serve

The smaller model may be preferable — **largest is not automatically best.**

SECTION 04 · FULL FINE-TUNING

Full Fine-Tuning — and Why It Gets Expensive

PROBLEM · RESOURCE COST

DEFINITION

In **full fine-tuning**, essentially all model parameters are allowed to update during training.

A `7-billion`-parameter model updated at full fine-tuning means updating approximately all `7 billion` parameters — maximum flexibility, but a major problem: **Resource Cost**.

TRAINING MEMORY — NOT ONLY WEIGHTS

Training does not only store model weights — it can also require memory for:

- Gradients
- Optimizer states
- Activations
- Temporary tensors

BASE MODEL — LAYER BY LAYER

- Layer 1 → Trainable
- Layer 2 → Trainable
- Layer 3 → Trainable
- . . .
- Layer N → Trainable

FP16 parameter storage alone = 7B × 2 bytes ≈ 14 GB

But full training additionally requires gradients, optimizer states, activations and framework overhead — actual training memory can be many times larger.

! Model Weight Size ≠ Total Training Memory

COMMON MISCONCEPTION

"A 7B model in FP16 is around 14 GB, so a 16 GB GPU should train it." → **Not necessarily** — training needs much more memory than storing the weights alone.

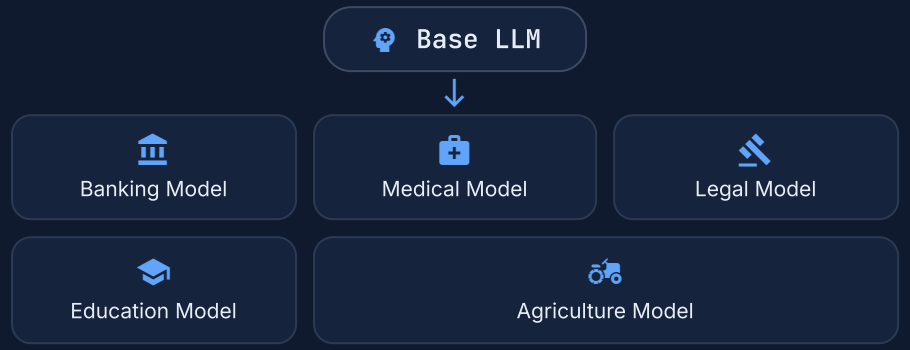
SECTION 04 · THE PEFT IDEA

Do We Need to Change Every Parameter?

from five full models to one smart idea

FULL FINE-TUNING AT SCALE

A company needs five specialized models:



Full fine-tuning here means potentially **five complete sets of updated model weights**, which increases:

Storage ↑ · Training cost ↑ · Model management complexity ↑

Do we really need to change all billions of model parameters for every small specialization?

THE PEFT IDEA

Large Base Model → Keep most parameters frozen → Train only a small number → Task-Specific Adaptation

= **PEFT** (Parameter-Efficient Fine-Tuning)

"frozen" means those parameters are not updated by gradient descent during fine-tuning — only selected/additional trainable parameters are optimized.

Full Fine-Tuning
= all / most weights trainable

PEFT
= base weights mostly frozen + small trainable component

TEXTBOOK ANALOGY

Imagine a 500-page textbook

Full fine-tuning rewrites many or all 500 pages.

PEFT keeps the textbook and adds a short task-specific supplement — the underlying book remains available.

SECTION 04 · THE PEFT FAMILY

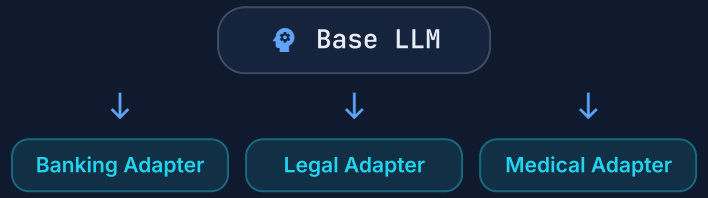
Why PEFT Matters — and a Preview of LoRA

adapt more, train less

WHY TEAMS CHOOSE PEFT — SIX BENEFITS

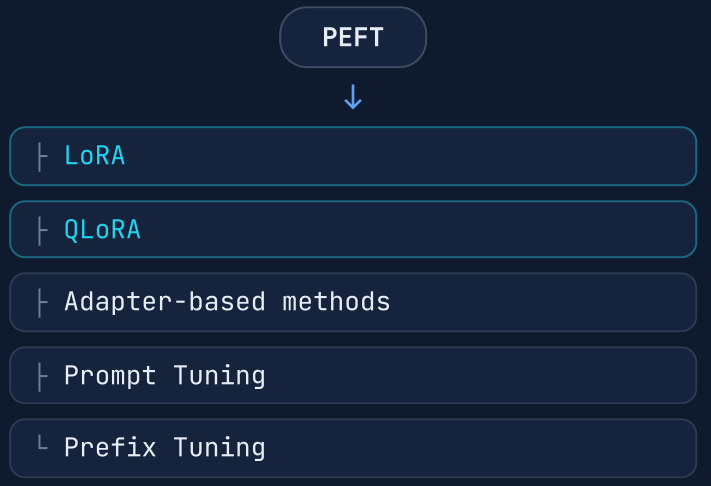
- Lower GPU memory
- Fewer trainable parameters
- Faster experimentation
- Smaller task-specific checkpoints
- Easier adaptation of large models
- Multiple adapters can share one base model

ONE BASE, MANY ADAPTERS



— instead of maintaining three entirely separate full models.

THE PEFT FAMILY TREE



★ For this course: **LoRA and QLoRA**.

LORA — FIRST GLANCE

LoRA = Low-Rank Adaptation

Core idea:

Base Model Weights → Frozen

Small Trainable Matrices → Learned

— instead of changing the entire large model.

"Imagine having a model with billions of parameters and discovering that we can adapt it by training only a tiny fraction of additional parameters."

Then stop — let the curiosity remain for **Lecture 3**.

SECTION 04 · CLASSROOM PRACTICE

Practice: From Requirements to Decisions

worked example first · then argue

COMPLETE DECISION EXAMPLE

University AI Assistant

"Answer formally."
→ **Prompt Engineering**

"Tell students today's/current examination deadlines."
→ **RAG**

"Classify every helpdesk request using our internal taxonomy."
→ **Fine-Tuning**

"Use current regulations AND internal classification behavior."
→ **RAG + Fine-Tuning**

CLASSROOM ACTIVITY — SIX SCENARIOS

argue in pairs · then reveal

1 *"Rewrite the paragraph in simple English."*
→ **Prompt engineering**

2 *"Answer questions from the latest HR policy PDF."*
→ **RAG**

3 *"Classify 50,000 daily customer complaints using 25 company-specific categories."*
→ **Fine-tuning is a strong candidate**

4 *"Tell me today's share price."*
→ **External real-time retrieval/API, not fine-tuning**

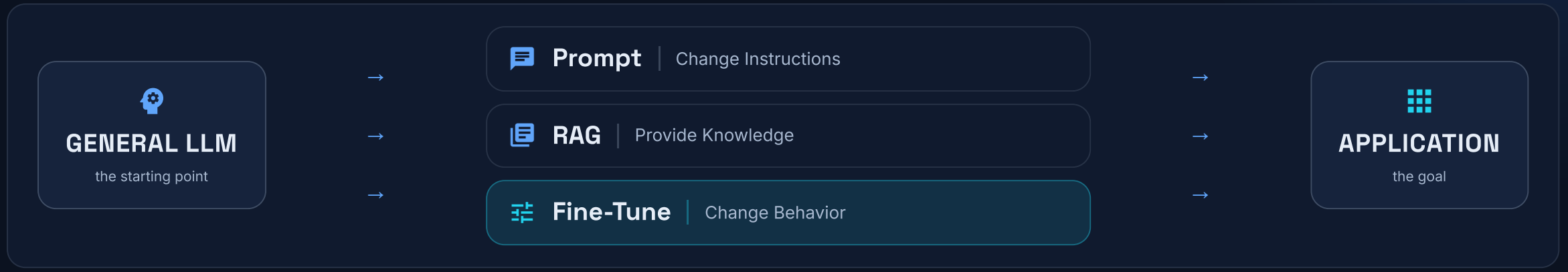
5 *"Model must always generate medical reports in the hospital's required schema."*
→ **Fine-tuning may be useful**

6 *"Generate hospital report format using current patient records."*
→ **Combination of controlled data retrieval/context + task-specific behavior**

SECTION 04 · LECTURE SUMMARY

What Did We Learn Today?

one decision map · eleven takeaways



Prompt → Input changes · RAG → Context changes · Fine-Tuning → Parameters change — *probably the single most useful conceptual comparison in Lecture 2.*

ELEVEN TAKEAWAYS

- ✓ Fine-tuning starts with an existing pre-trained LLM
- ✓ It adapts model behavior for a specific task/domain
- ✓ Fine-tuning changes model parameters
- ✓ Prompt engineering changes instructions, not parameters
- ✓ RAG supplies external information without retraining the model
- ✓ Fine-tuning requires high-quality task-specific examples
- ✓ Supervised fine-tuning uses known input–output examples
- ✓ Train, validation and test data have different purposes
- ✓ Full fine-tuning updates most/all parameters
- ✓ Full fine-tuning can be expensive
- ★ PEFT reduces the number of trainable parameters

SECTION 04 · CLOSING

Self-Test & The Road to Lecture 3

11 questions · answer first, then check

RECAP Base LLM + Task-Specific Dataset → Tokenize → Forward Pass → Prediction → Loss → Backpropagation → Parameter Updates → Repeat Over Dataset → Specialized Model

Q1 What is fine-tuning?
→ Further training a pre-trained model using task/domain-specific data.

Q2 Does prompt engineering change model weights?
→ No.

Q3 Does RAG normally change model weights?
→ No.

Q4 What does RAG change?
→ The information/context provided to the LLM during inference.

Q5 What does fine-tuning primarily change?
→ Model behavior through parameter updates.

Q6 Which data does the model learn from?
→ Training set.

Q7 Which dataset is used to select configurations?
→ Validation set.

Q8 Which dataset is reserved for final evaluation?
→ Test set.

Q9 What is full fine-tuning?
→ Most or all model parameters are trainable.

Q10 Why is full fine-tuning expensive?
→ Enormous parameters plus weights, gradients, optimizer states and activations in memory.

Q11 What is PEFT?
→ Parameter-Efficient Fine-Tuning — adapting a model while training only a relatively small number of parameters.

Model = 7 Billion Parameters

full fine-tuning → High Memory · High Compute · High Cost

"Can we specialize the LLM by training only a very small fraction of parameters?"

→ YES PEFT → LoRA → QLoRA — next lecture.